The KV cache and PagedAttention

How an LLM server remembers what it has already read, and where it keeps it

Carlos Cotrini

ETH Zurich

7 October 2026

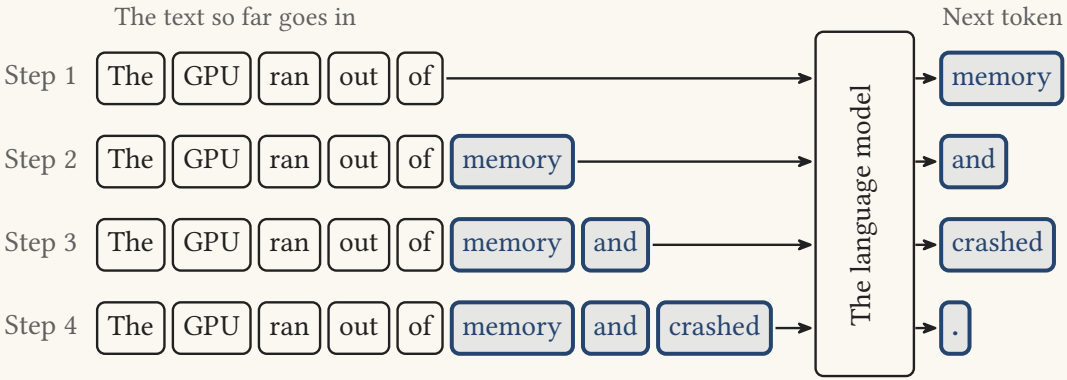
1

Part 1

The KV cache

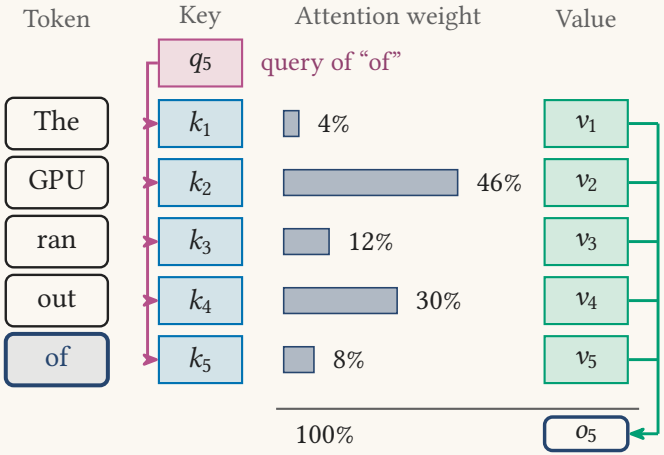
Keep what never changes, and compute only what is new

A language model writes one token at a time



Each step reads the whole text so far and answers with one token (a word or a piece of a word). That token, in blue, is appended and the model runs again.

Attention is a lookup in a table

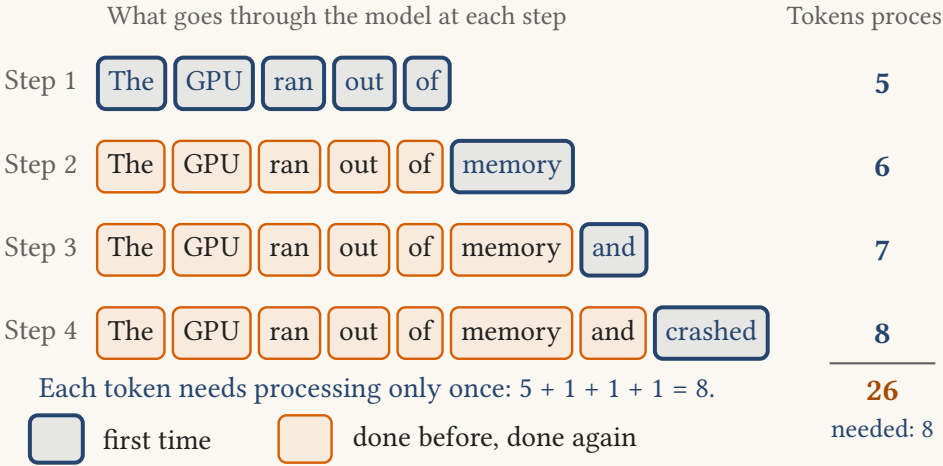


1. Every token so far, the newest one included, offers a key (how to find it) and a value (what it gives).

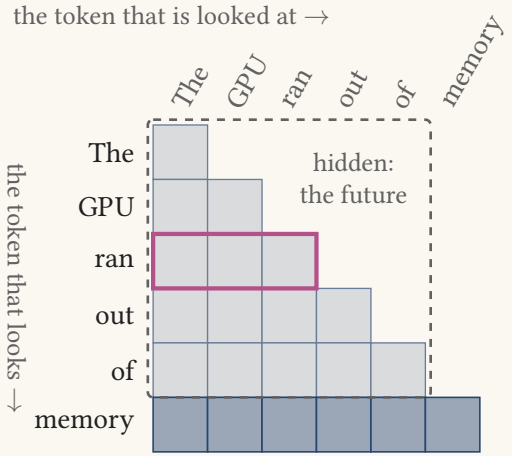
2. The query of the newest token meets every key: a good match gives a large weight (made up here).

3. The output mixes the values with those weights.

Without a cache, every step repeats the old work



A new token never changes the old ones



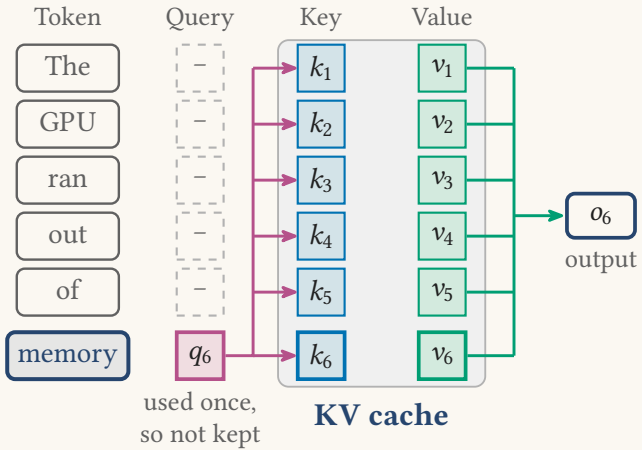
Each row is one token. It looks at itself and the tokens before it, never at later ones. Row 3, “ran”, is outlined.

A new token adds **one row**. The old rows are exactly as before.

The same triangle holds in every layer: everything computed for tokens 1 to 5 is still valid.

So keep the old keys and values: memoization. That store is the KV cache.

With a cache, a step computes only the new token



1. The cache holds the keys and values of all earlier tokens.

2. New: its query, key and value. Reused: 5 rows.

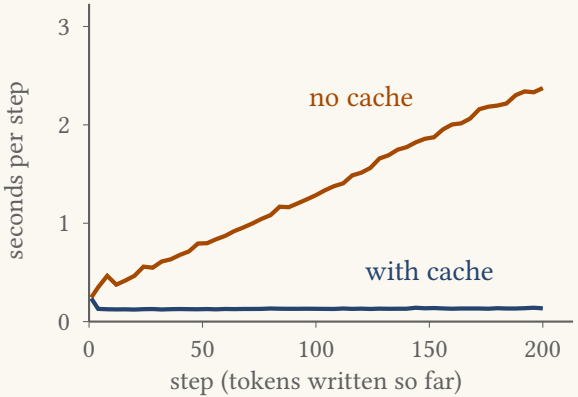
3. Its key and value are appended to the cache.

4. Its query is compared with every key in the cache.

5. The values are mixed with the attention weights: this token's output.

The cache changes the speed, not the answer

Qwen2.5-0.5B (a small open model) on a laptop CPU: 200 tokens after “The GPU ran out of”



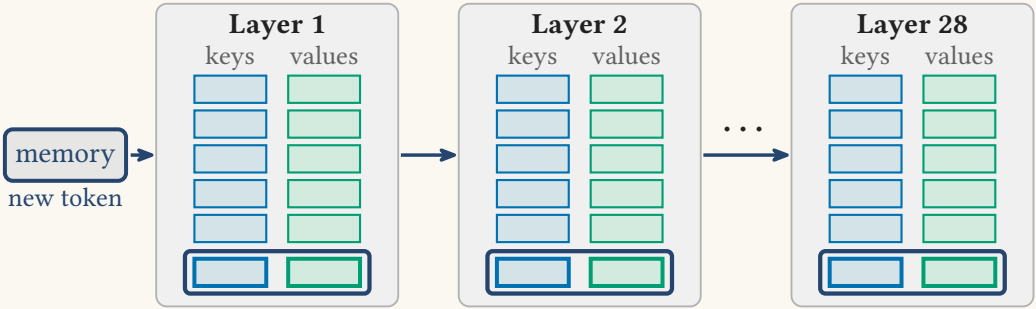
Without the cache, each step takes longer than the last. Total: 268 s.

With the cache, every step costs about the same. Total: 26 s.

Same 200 tokens in both runs.

Appending one more token changes the old keys and values by about 0.00001 (float32 rounding; the keys reach 215), in all 24 layers.

Every layer keeps its own cache

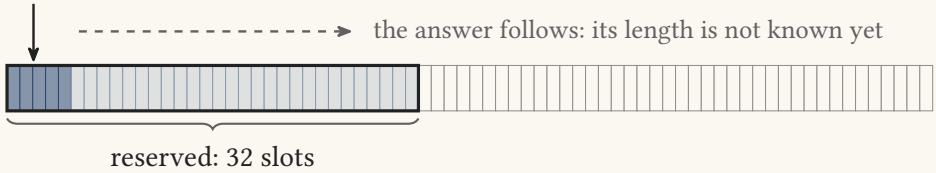


Each layer computes keys and values from the output of the previous layer, so each layer keeps its own cache: 28 of them in Qwen2.5-7B (the small model of the previous slide has 24). A new token adds one row to every one.

The server reserves room for the longest case

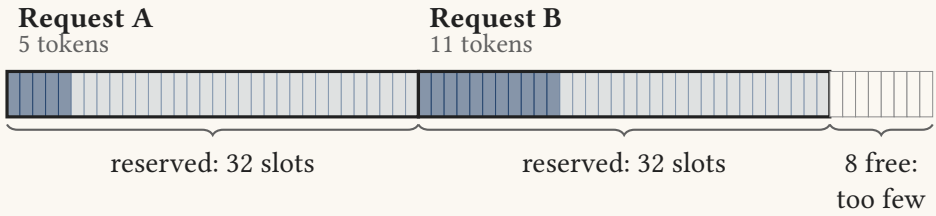
Toy cache memory: 72 slots. A slot holds one token's key and value (all layers).

Request A "The GPU ran out of"



1. A server keeps a cache per request. Request A arrives: its prompt takes 5 slots.
2. Its answer will take more, but the model decides when to stop.
3. The server accepts requests of at most 32 tokens, so it reserves 32 slots.
4. It reserves them in one piece: frameworks store a cache as one contiguous tensor.

Two requests fill the memory

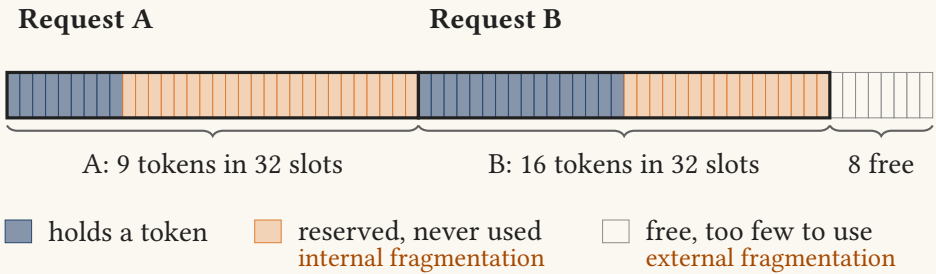


1. Request B, with a prompt of 11 tokens, reserves 32 slots too.

2. 8 slots are left, too few for a third request: it needs 32 in one piece.

Most reserved slots are never used

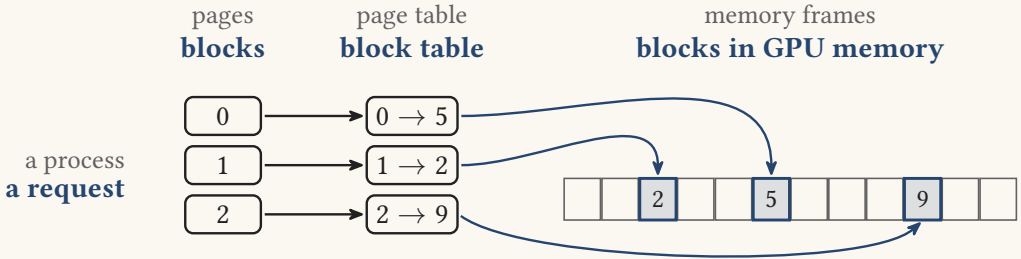
Later, both answers have ended.



25 of 72 slots hold tokens: 35%

(In the PagedAttention paper, only 20.4% to 38.2% of the cache memory held tokens, in its own versions of Orca, an earlier server.)

Operating systems solved this with paging



PagedAttention: same idea, two vocabularies

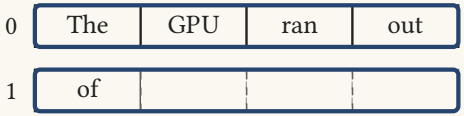
a byte is a token (its key and value)

a page is a block of tokens

a process is a request

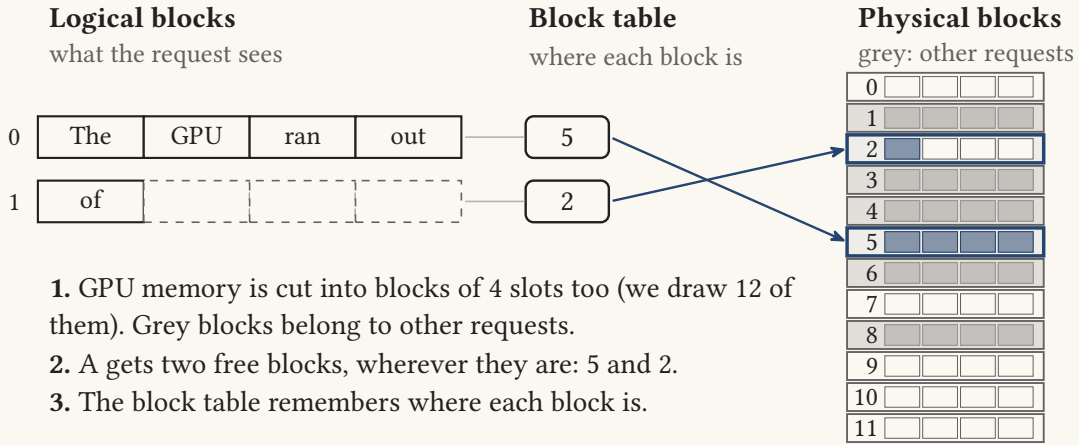
A request's slots are grouped in blocks

Logical blocks
what the request sees



1. The slots of a request are grouped in blocks. In our toy a block has 4 slots (real servers: 16).
2. Request A has 5 tokens: block 0 is full.
3. Block 1 holds one token and has 3 free slots.

Blocks can sit anywhere in GPU memory

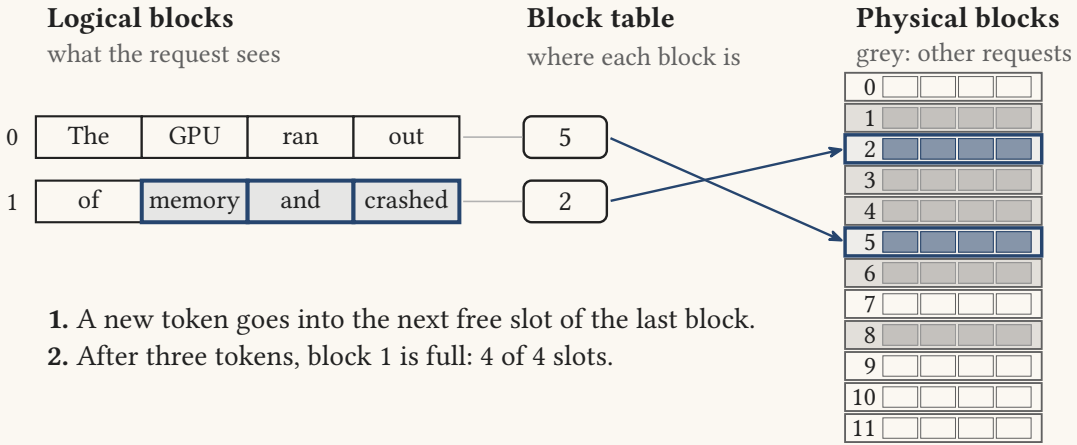


1. GPU memory is cut into blocks of 4 slots too (we draw 12 of them). Grey blocks belong to other requests.

2. A gets two free blocks, wherever they are: 5 and 2.

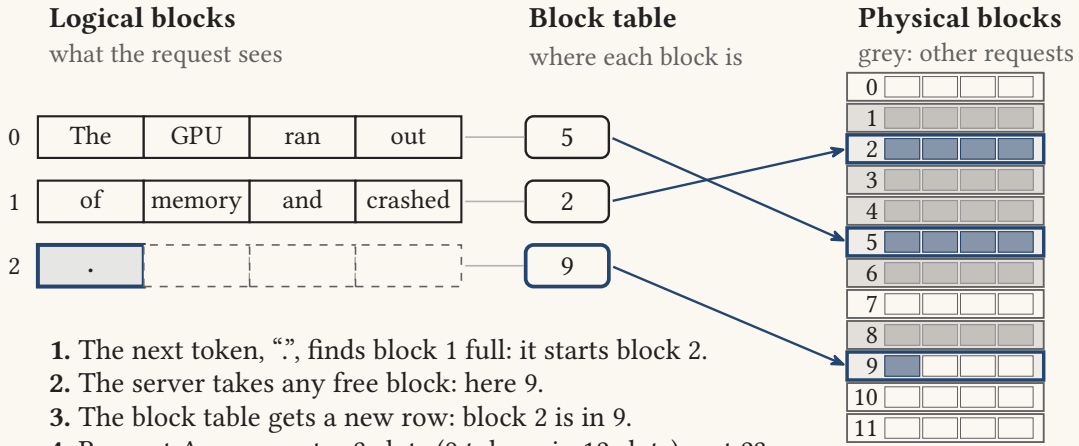
3. The block table remembers where each block is.

New tokens fill the last block



1. A new token goes into the next free slot of the last block.
2. After three tokens, block 1 is full: 4 of 4 slots.

A new block is taken when the last is full



1. The next token, ".", finds block 1 full: it starts block 2.
2. The server takes any free block: here 9.
3. The block table gets a new row: block 2 is in 9.
4. Request A now wastes 3 slots (9 tokens in 12 slots), not 23.

Two ideas to remember

The KV cache

- The keys and values of old tokens never change, so keep them.
- Each step computes only the new token, so steps stay fast.
- The price is memory: every token of every running request keeps its keys and values until the request ends.

PagedAttention

- Do not reserve for the longest answer. Cut each cache into small blocks.
- A block table maps logical blocks to physical ones, like a page table.
- Only the last block of a request can have free slots, and any free block fits.

Together: keep the old keys and values, and keep them in blocks, so that more requests fit in memory.